

实验二：灰度图像的直方图规定化处理

姓名：梁信、高居博、赵夕涵、刘嘉乐、黎玥含、赵欣瑶、罗雅丹、蓝浩宇

1 实验目标

本实验的目标是掌握灰度图像的直方图处理方法，具体包括：

1. 理解直方图均衡化（Histogram Equalization）的原理与实现；
2. 理解直方图规定化（Histogram Specification / Matching）的原理与实现；
3. 以正弦函数 $y = \sin(x)$, $x \in [0, \pi]$ 作为目标直方图，对给定灰度图像进行规定化处理；
4. 通过编程实现上述算法，并对处理结果进行分析。

2 方法描述

2.1 直方图与归一化直方图

设灰度图像的灰度级范围为 $[0, L - 1]$ （本实验中 $L = 256$ ），图像共有 N 个像素。灰度级 k 出现的次数为 n_k ，则归一化直方图（概率密度函数的离散近似）为：

$$H(k) = p(r_k) = \frac{n_k}{N}, \quad k = 0, 1, \dots, L - 1 \quad (1)$$

2.2 直方图均衡化

直方图均衡化的目标是将原图直方图变换为近似均匀分布。其变换函数基于累积分布函数（CDF）：

$$T_{se}(k) = \text{round}\left((L - 1) \sum_{i=0}^k H_s(i)\right) = \text{round}\left(255 \sum_{i=0}^k H_s(i)\right) \quad (2)$$

其中 H_s 为原图直方图。对原图每个像素灰度值 r 施加变换 $s = T_{se}(r)$ 即可得到均衡化图像 I_{se} 。

2.3 目标直方图生成

本实验给定目标直方图为正弦函数形状：

$$H'_{dd}(k) = \sin\left(\frac{k}{255} \cdot \pi\right), \quad k = 0, 1, \dots, 255 \quad (3)$$

归一化后得到目标概率分布：

$$H_{dd}(k) = \frac{H'_{dd}(k)}{\sum_{i=0}^{255} H'_{dd}(i)} \quad (4)$$

2.4 直方图规定化（SML 单映射法）

直方图规定化的核心思想是：通过均衡化作为”桥梁”，将原图的灰度分布映射到目标分布。具体步骤如下：

步骤 1： 计算原图直方图 H_s 的均衡化变换函数：

$$T_s(k) = \text{round} \left(255 \sum_{i=0}^k H_s(i) \right) \quad (5)$$

步骤 2： 计算目标直方图 H_{dd} 的均衡化变换函数：

$$G(k) = \text{round} \left(255 \sum_{i=0}^k H_{dd}(i) \right) \quad (6)$$

步骤 3： 对原图每个灰度级 k ，在目标 CDF 映射中查找最近匹配：

$$T_d(k) = \arg \min_{j \in [0, 255]} |T_s(k) - G(j)| \quad (7)$$

步骤 4： 对原图每个像素灰度值 r 施加变换 $s = T_d(r)$ ，得到规定化目标图像 I_d 。

3 实验结果

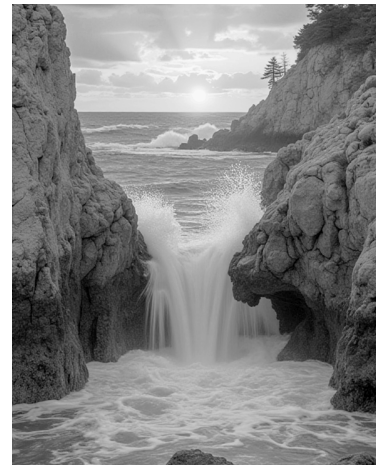
3.1 原始图像与处理结果



(a) 原图 I_s



(b) 均衡化图像 I_{se}



(c) 规定化图像 I_d

图 1: 原始图像与处理结果对比

3.2 任务 3: 原图直方图 H_s

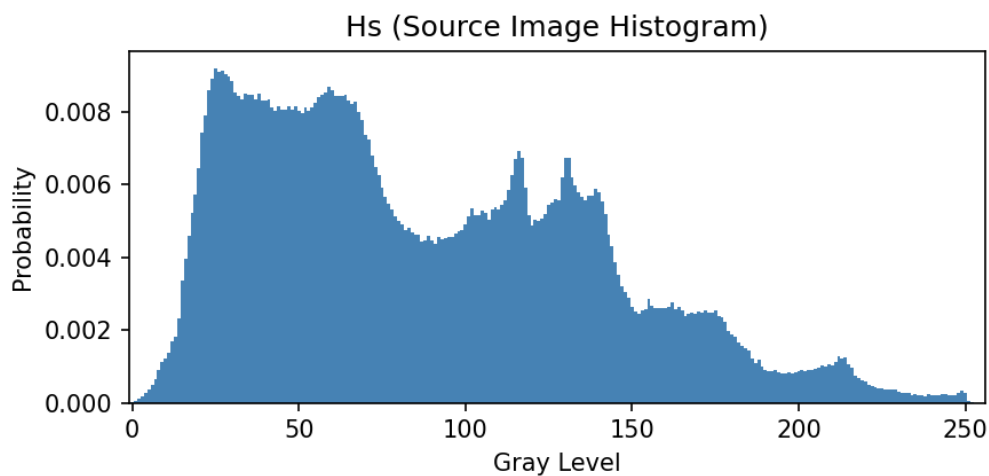


图 2: 原图直方图 H_s

从图中可以看出，原图灰度值集中在中低灰度区间（约 20–140），说明原图整体偏暗，对比度较低。

3.3 任务 5: 均衡化变换函数 T_{se}

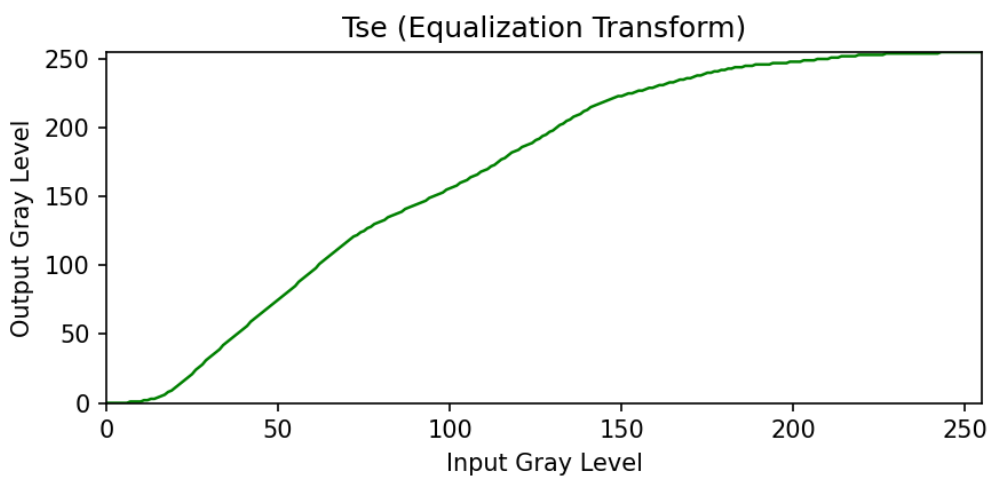


图 3: 均衡化变换函数 T_{se}

均衡化变换函数为 CDF 的缩放映射，呈单调递增的 S 形曲线。在原图直方图密集的区域斜率较大（拉伸），稀疏区域斜率较小（压缩）。

3.4 任务 6: 均衡化图像直方图 H_{se}

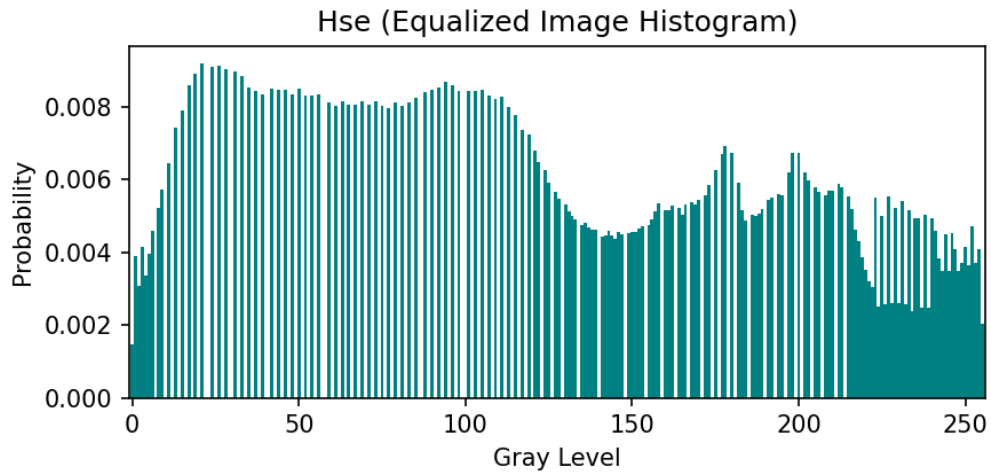


图 4: 均衡化图像直方图 H_{se}

均衡化后的直方图在整个灰度范围 $[0, 255]$ 上近似均匀分布，验证了均衡化算法的正确性。由于离散灰度级的限制，分布不是严格均匀的。

3.5 目标直方图 H_{dd} (正弦函数)

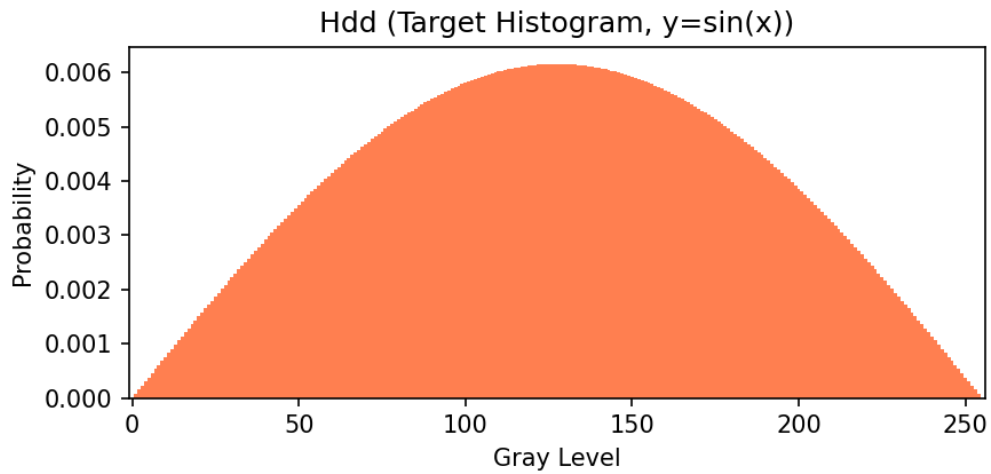


图 5: 目标直方图 H_{dd} ($y = \sin(x), x \in [0, \pi]$)

目标直方图呈正弦曲线形状，在中间灰度级 ($k = 128$ 附近) 概率最大，两端趋近于零。

3.6 任务 2：规定化变换函数 T_d

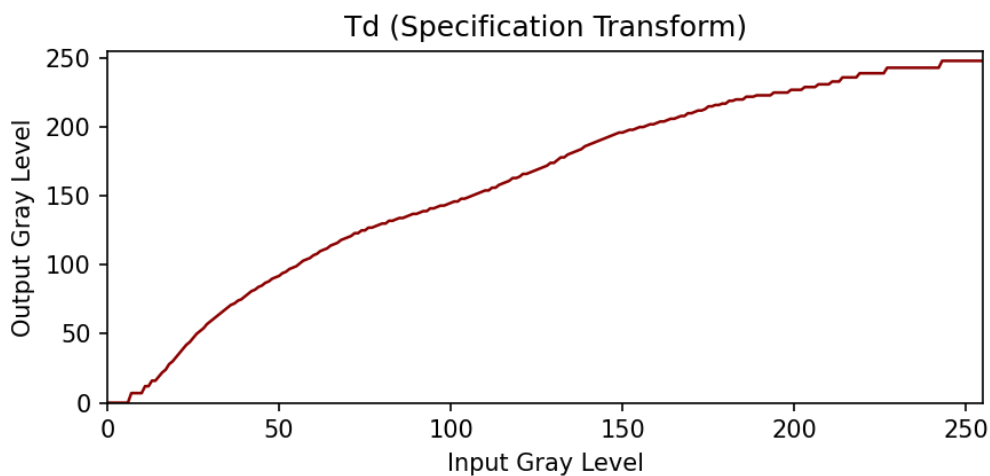


图 6: 规定化变换函数 T_d

规定化变换函数通过 SML 单映射法得到。可以看出在中间灰度区域映射较为线性，而在高灰度区出现阶梯状跳跃，这是因为正弦目标直方图在两端概率较小，多个原始灰度级被映射到同一目标灰度级。

3.7 任务 8：规定化图像直方图 H_d

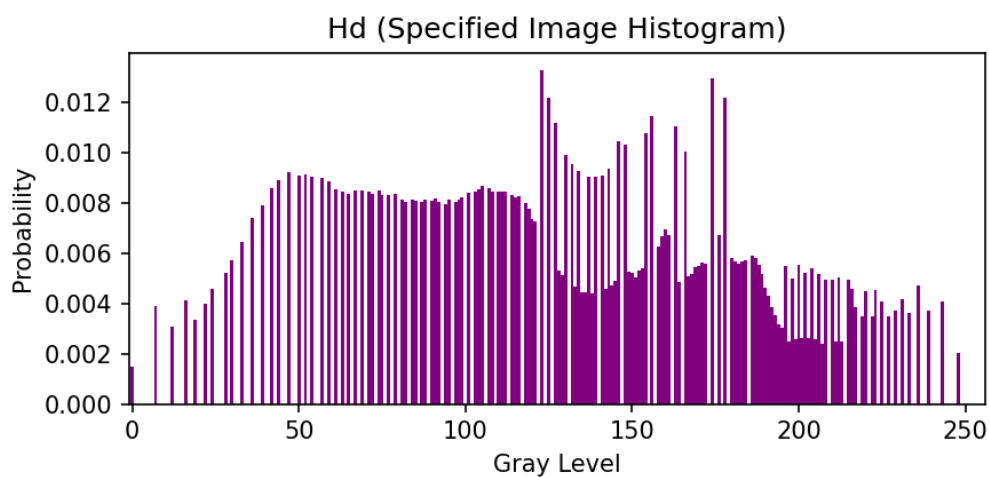


图 7: 规定化图像直方图 H_d

规定化后的图像直方图整体呈现正弦函数的包络形状，中间灰度级概率较高，两端较低，与目标直方图 H_{dd} 的趋势一致。由于离散映射的限制，某些灰度级概率为零（出现间隙），但整体分布已较好地逼近目标。

3.8 任务 7：规定化图像的均衡化变换函数 T_{de}

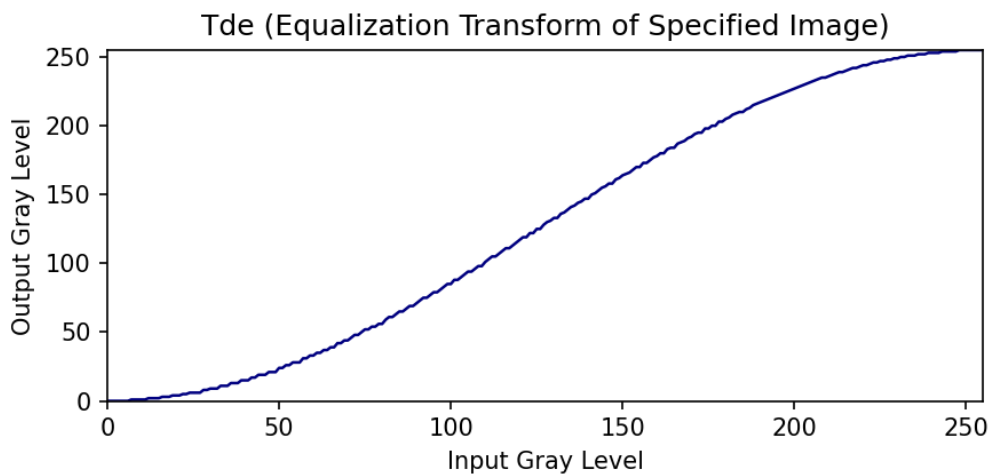


图 8: 规定化图像的均衡化变换函数 T_{de}

规定化图像 I_d 的均衡化变换函数呈 S 形曲线，与目标直方图 H_{dd} 的 CDF 形状一致 ($1 - \cos(x)$ 的形态)，验证了规定化处理的正确性。

3.9 结果小结

任务	内容	输出文件	说明
1	规定化目标图像 I_d	d.bmp	直方图近似正弦分布
2	规定化变换函数 T_d	Td.txt	SML 单映射法求得
3	原图直方图 H_s	Hs.txt	灰度集中在中低区
4	均衡化图像 I_{se}	Ise.bmp	对比度显著增强
5	均衡化变换函数 T_{se}	Tse.txt	CDF 缩放映射
6	均衡化直方图 H_{se}	Hse.txt	近似均匀分布
7	I_d 的均衡化变换 T_{de}	Tde.txt	与目标 CDF 一致
8	规定化直方图 H_d	Hd.txt	近似正弦包络

表 1: 实验输出文件汇总

4 附录：函数代码

以下为除 HXLBMPIFILE 类代码外的所有实现函数 (main.cpp)。

```

1 #include "hxlbmpfile.h"
2 #include <math.h>
3 #include <string.h>
4
5 #ifndef M_PI
6 #define M_PI 3.14159265358979323846
7 #endif

```

```

8
9 // Func1: Calc image histogram
10 // Input: image pointer; Output: normalized H[256]
11 void CalcHistogram(HXLBMPFILE* bmp, float H[256]) {
12     memset(H, 0, 256 * sizeof(float));
13     int total = bmp->iImageh * bmp->iImagew;
14     for (int i = 0; i < bmp->iImageh; i++)
15         for (int j = 0; j < bmp->iImagew; j++)
16             H[bmp->pDataAt(i)[j]] += 1.0f;
17     for (int k = 0; k < 256; k++)
18         H[k] /= (float)total;
19 }
20
21 // Func2: Histogram equalization transform
22 // Input: normalized H; Output: mapping table T[256]
23 void CalcEqualizationTransform(float H[256], BYTE T[256]) {
24     float cdf[256];
25     cdf[0] = H[0];
26     for (int i = 1; i < 256; i++)
27         cdf[i] = cdf[i - 1] + H[i];
28     for (int i = 0; i < 256; i++)
29         T[i] = (BYTE)(cdf[i] * 255.0f + 0.5f);
30 }
31
32 // Func3: Generate target histogram (y=sin(x), x in [0,pi])
33 void GenerateTargetHistogram(float Hdd[256]) {
34     float sum = 0.0f;
35     for (int k = 0; k < 256; k++) {
36         float x = (float)k / 255.0f * (float)M_PI;
37         Hdd[k] = sinf(x);
38         sum += Hdd[k];
39     }
40     for (int k = 0; k < 256; k++)
41         Hdd[k] /= sum;
42 }
43
44 // Func4: Histogram specification (SML method)
45 void CalcSpecificationTransform(float Hs[256],
46                                float Hdd[256], BYTE Td[256]) {
47     BYTE Ts[256], Tdd[256];
48     CalcEqualizationTransform(Hs, Ts);
49     CalcEqualizationTransform(Hdd, Tdd);
50     for (int k = 0; k < 256; k++) {
51         int minDiff = 999, bestJ = 0;
52         for (int j = 0; j < 256; j++) {
53             int diff = abs((int)Ts[k] - (int)Tdd[j]);
54             if (diff < minDiff) {
55                 minDiff = diff;
56                 bestJ = j;
57             }

```

```

58     }
59     Td[k] = (BYTE)bestJ;
60 }
61 }
62
63 // Func5: Save array to text file
64 void SaveArrayToTxt_Float(float arr[256],
65                          const char* filename) {
66     FILE* fp = fopen(filename, "w");
67     if (!fp) return;
68     for (int i = 0; i < 256; i++)
69         fprintf(fp, "%d\t%f\n", i, arr[i]);
70     fclose(fp);
71 }
72 void SaveArrayToTxt_Byte(BYTE arr[256],
73                         const char* filename) {
74     FILE* fp = fopen(filename, "w");
75     if (!fp) return;
76     for (int i = 0; i < 256; i++)
77         fprintf(fp, "%d\t%d\n", i, (int)arr[i]);
78     fclose(fp);
79 }
80
81 // Func6: Apply gray-level mapping
82 void ApplyTransform(HXLBMPFILE* src, BYTE T[256],
83                   HXLBMPFILE* dst) {
84     dst->iImagew = src->iImagew;
85     dst->iImageh = src->iImageh;
86     dst->iYRGBnum = 1;
87     if (!dst->IspImageDataOk()) return;
88     for (int k = 0; k < 256; k++) {
89         dst->rgbPalette[k].rgbRed = k;
90         dst->rgbPalette[k].rgbGreen = k;
91         dst->rgbPalette[k].rgbBlue = k;
92         dst->rgbPalette[k].rgbReserved = 0;
93     }
94     for (int i = 0; i < src->iImageh; i++)
95         for (int j = 0; j < src->iImagew; j++)
96             dst->pDataAt(i)[j] = T[src->pDataAt(i)[j]];
97 }
98
99 // main
100 int main() {
101     HXLBMPFILE bmpIs;
102     if (!bmpIs.LoadBMPFile((char*)"b8gray.bmp"))
103         return 1;
104
105     float Hs[256], Hdd[256], Hse[256], Hd[256];
106     BYTE Tse[256], Td[256], Tde[256];
107

```



```
108     CalcHistogram(&bmpIs, Hs);
109     SaveArrayToTxt_Float(Hs, "Hs.txt");
110
111     CalcEqualizationTransform(Hs, Tse);
112     SaveArrayToTxt_Byte(Tse, "Tse.txt");
113
114     HXLBMPFILE bmpIse;
115     ApplyTransform(&bmpIs, Tse, &bmpIse);
116     bmpIse.SaveBMPFile((char*)"Ise.bmp");
117
118     CalcHistogram(&bmpIse, Hse);
119     SaveArrayToTxt_Float(Hse, "Hse.txt");
120
121     GenerateTargetHistogram(Hdd);
122     SaveArrayToTxt_Float(Hdd, "Hdd.txt");
123
124     CalcSpecificationTransform(Hs, Hdd, Td);
125     SaveArrayToTxt_Byte(Td, "Td.txt");
126
127     HXLBMPFILE bmpId;
128     ApplyTransform(&bmpIs, Td, &bmpId);
129     bmpId.SaveBMPFile((char*)"d.bmp");
130
131     CalcHistogram(&bmpId, Hd);
132     SaveArrayToTxt_Float(Hd, "Hd.txt");
133
134     CalcEqualizationTransform(Hd, Tde);
135     SaveArrayToTxt_Byte(Tde, "Tde.txt");
136
137     return 0;
138 }
```